



1. BDD EMPLEADA PARA ADMINISTRAR EL NEGOCIO DE LA EMPRESA ILAP

1. BDD EMPLEADA PARA ADMINISTRAR EL NEGOCIO DE LA EMPRESA ILAP.....	1
1.1. Transparencia de distribución para la instrucción select.....	2
1.1.1. Creación de sinónimos.....	2
1.1.1.1. Creación de sinónimos para tablas replicadas.....	2
Ejemplo.....	2
1.1.1.2. Validación de sinónimos.....	3
Ejemplo.....	3
1.1.2. Creación de vistas.....	4
1.1.2.1. Vistas para tablas globales sin columnas BLOB/CLOB.....	4
Ejemplo.....	4
Ejemplo.....	5
1.1.2.3. Creación de tablas temporales para acceso a datos BLOB/CLOB.....	6
Ejemplo.....	7
1.1.2.4. Creación de funciones para acceso a datos CLOB/BLOB.....	8
Ejemplo.....	8
Ejemplo.....	10
1.1.2.6. Ejecución de scripts de creación de vistas.....	11
Ejemplo.....	12
1.2. Transparencia de distribución para instrucciones DML.....	13
1.2.1. Creación de triggers para tablas con esquema de fragmentación.....	13
1.2.2. Creación de triggers para tablas replicadas.....	14
1.2.2.1. Replicación síncrona de tablas.....	14
Ventajas.....	14
Desventajas.....	15
Ejemplo.....	15
Ejemplo.....	17

1.1. Transparencia de distribución para la instrucción select

El siguiente paso a desarrollar es la implementación de transparencia. Para efectos del proyecto, se realizará únicamente transparencia para las instrucciones `select`, `insert` y `delete`.

1.1.1. Creación de sinónimos

- Crear un archivo por sitio llamado `s-04-ilap-<pdb>-sinonimos.sql`. Ejemplo: `s-04-ilap-jrc-s1-sinonimos.sql`
- Incluir los sinónimos requeridos por PDB los cuales serán empleados para implementar transparencia de localización.
- El script deberá conectarse a la PDB y crear los sinónimos correspondientes.
- Observar en el siguiente ejemplo, la convención a emplear: `<nombre_global>_f<n>` Donde N es el número de fragmento. A partir de este punto se oculta la información de ubicación de los fragmentos. Por ejemplo, se oculta `jrc_s1`, `jrc_s2`, etc.
- Observar que también se crean sinónimos para fragmentos locales. Por ejemplo: para el nodo `jrcbdd_s1` se crea el sinónimo `usuario_f1` a partir de `usuario_f1_jrc_s1`

1.1.1.1. Creación de sinónimos para tablas replicadas

- Para el caso de las tablas que serán replicadas, también deberán contar con sus sinónimos: 3 remotos y uno local. Emplear la notación `<nombre_global>_r<n>`
- En este caso "N" no significa el número de fragmento, representa el número de réplica. Al contar con 4 PDBs, se tendrá un total de 4 réplicas. Por convención la primera réplica será considerada la réplica local. Por ejemplo, para la tabla `tipo_monitor_r_jrc_s1` sus sinónimos serán `tipo_monitor_r1`, `tipo_monitor_r2`, `tipo_monitor_r3` y `tipo_monitor_r4`. El primer sinónimo apuntará a la tabla local `tipo_monitor_r_jrc_s1` y los otros 3 apuntarán a las réplicas remotas.
- Las copias manuales de tablas no requieren sinónimos.

Ejemplo

Sinónimos para `jrcbdd_s1`.

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:    Creación de sinónimos en Los 4 nodos.

--sucursal
create or replace synonym sucursal_f1 for sucursal_f1_jrc_s1;
create or replace synonym sucursal_f2 for sucursal_f2_jrc_s2@jrcbdd_s2;
create or replace synonym sucursal_f3 for sucursal_f3_arc_s1@arcbdd_s1;
create or replace synonym sucursal_f4 for sucursal_f4_arc_s2@arcbdd_s2;
```

```
--completar
```

1.1.1.2. Validación de sinónimos

Para verificar que los sinónimos y fragmentos han sido creados correctamente en sus correspondientes sitios, crear un script llamado `s-04-ilap-valida-sinonimos.sql`. El script mostrará un conteo del número de registros existente principalmente para validar que el sinónimo esté creado correctamente.

Cabe mencionar que el manejador no verifica si la tabla realmente existe durante la creación del sinónimo, por lo que este script será útil para detectar posibles errores.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación:  dd/mm/yyyy
--@Descripción:     Script de validación de sinónimos
```

Prompt validando sinónimos para sucursal

```
select
  (select count(*) from sucursal_f1) as sucursal_f1,
  (select count(*) from sucursal_f2) as sucursal_f2,
  (select count(*) from sucursal_f3) as sucursal_f3,
  (select count(*) from sucursal_f4) as sucursal_f4
from dual;
```

```
-- completar
```

1.1.1.3. Ejecución de scripts para sinónimos

Crear un solo script llamado `s-04-ilap-main-sinonimos.sql`. El script se deberá conectar a cada PDB y ejecutar los scripts creados anteriormente. Notar que el script de validación se ejecuta en cada una de las PDBs para confirmar su correcta creación.

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación:  dd/mm/yyyy
--@Descripción:     Creación de sinónimos - main
clear screen
whenever sqlerror exit rollback;

prompt =====
prompt Creando sinónimos para jrcbdd_s1
prompt =====
```

```

connect ilap_bdd/ilap_bdd@jrcbdd_s1
@s-04-ilap-jrc-s1-sinonimos.sql
@s-04-ilap-valida-sinonimos.sql

prompt =====
prompt creando sinónimos para jrcbdd_s2
prompt =====
connect ilap_bdd/ilap_bdd@jrcbdd_s2
@s-04-ilap-jrc-s2-sinonimos.sql
@s-04-ilap-valida-sinonimos.sql

prompt =====
prompt creando sinónimos para arcbdd_s1
prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s1
@s-04-ilap-arc-s1-sinonimos.sql
@s-04-ilap-valida-sinonimos.sql

prompt =====
prompt creando sinónimos para arcbdd_s2
prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s2
@s-04-ilap-arc-s2-sinonimos.sql
@s-04-ilap-valida-sinonimos.sql

prompt Listo!

```

1.1.2. Creación de vistas

1.1.2.1. Vistas para tablas globales sin columnas BLOB/CLOB

- Crear un script llamado `s-05-ilap-vistas.sql`. El script contendrá la definición de las vistas que se van a crear en cada sitio.
- No incluir en este script las vistas que corresponden a tablas con datos BLOB/CLOB. Esto sin importar el sitio en el que se encuentre. Las vistas para estas tablas se crearán en la siguiente sección.

Ejemplo

```

--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:    Creación de vistas que no requieren manejo de BLOBs.

```

```
--sucursal
create or replace view sucursal as
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f1
union all
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f2
union all
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f3
union all
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f4;

--completar
```

- Observar que se listan los nombres de cada columna. Evitar el uso de “*” ya que el orden en el que se definen los atributos en cada fragmento puede cambiar y generar inconsistencias y/o errores. Hacer uso de `union all`.
- Observar que este código se puede ejecutar en las 4 PDBs ya que el código es exactamente el mismo para todos los sitios.

1.1.2.2. Vistas para tablas replicadas

- Para implementar el mecanismo de replicación es necesario crear una vista para cada una de las tablas que serán replicadas.
- Para una tabla que cuenta con su propio esquema de fragmentación, su vista define una consulta que permite recuperar todos los datos (expresión de reconstrucción). Sin embargo, para una tabla replicada que no cuenta con esquema de fragmentación, la vista no incluye la expresión de reconstrucción.
- Debido a que en todos los sitios existirá exactamente la misma copia de datos (réplica), la vista únicamente deberá mostrar los datos de alguna de estas réplicas. La técnica más eficiente es apuntar a la copia local que anteriormente se estableció con el sinónimo `<nombre_tabla>_r1`. Es decir, la réplica `r1` apunta a la réplica local.
- Por lo anterior, en cada PDB se deberá crear una vista que apunte a su réplica local.

Con base a lo anterior, agregar la definición de las vistas para cada tabla replicada en el archivo `s-05-ilap-vistas.sql`.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:     Creación de vistas que no requieren manejo de BLOBs.
```

```
--sucursal
create or replace view sucursal as
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f1
union all
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f2
union all
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f3
union all
select sucursal_id,clave,es_taller,es_venta,nombre,latitud,longitud,url from
sucursal_f4;

--completar
```

1.1.2.3. Creación de tablas temporales para acceso a datos BLOB/CLOB

En prácticas anteriores se revisaron 2 técnicas para implementar transparencia de distribución para acceder a un dato BLOB/CLOB de forma remota. En este proyecto se emplea únicamente la estrategia 2 en la que se obtiene un dato binario a través de su Id. Para ello, se deberán crear los siguientes objetos adicionales:

- Tabla temporal para almacenar el dato BLOB/CLOB para transparencia de operaciones `select`. Se emplea la notación `ts_<nombre_global>_<num_fragmento>`. Por ejemplo, `ts_laptop_1` es una tabla temporal que se emplea para almacenar el dato BLOB que se obtiene del fragmento 1
- Tabla temporal para almacenar el dato BLOB/CLOB para transparencia de operaciones `insert`. Se emplea la notación `ti_<nombre_global>_<num_fragmento>`.
- Funciones encargadas de obtener un dato BLOB/CLOB de un sitio remoto. Se emplea la sintaxis `get_remote_<nombre_columna_blob>_f<numero_fragmento>_by_id`.
- Por ejemplo, la función `get_remote_foto_f1_by_id` será una función que obtiene el contenido del dato BLOB asociado a la columna foto del fragmento 1 (`foto_f1`).
- Notar que no se requiere crear otros objetos que fueron empleados en prácticas anteriores (objetos `type`, `table`) ya que se estará empleando la estrategia 2.
- Observar que estas tablas temporales y funciones son accedidas por prácticamente todos los nodos. Algunos nodos pudieran no requerirse ya que el dato BLOB/CLOB se encontrará localmente. Lo ideal sería crear estas funciones únicamente en los sitios donde se requieren, pero para evitar duplicidad de código y aumento de la complejidad de los scripts, se creará uno solo y será ejecutado en todos los sitios.

Para implementar estos objetos, realizar las siguientes acciones:

- Crear un script llamado `s-05-ilap-tablas-temporales.sql` . El script deberá contener todas las tablas temporales que requieran manejo de datos CLOB/BLOB tanto para operaciones `insert` como para operaciones `select`. El código es exactamente el mismo, solo varían por el nombre, pero se usarán para propósitos diferentes.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:     Creación de tablas temporales para manejo de BLOBs.
```

Prompt eliminando tablas en caso de existir

```
declare
  cursor cur_tablas is
    select table_name
    from user_tables
    where table_name in ('TI_LAPTOP_F1','TS_LAPTOP_F1')
    or table_name like 'T__SERVICIO_LAPTOP_F_';
begin
  for r in cur_tablas loop
    execute immediate 'drop table ' || r.table_name;
  end loop;
end;
/
```

Prompt tablas temporales para la tabla laptop

```
create global temporary table ti_laptop_f1(
  laptop_id number(10,0) constraint ti_laptop_f1_pk primary key,
  foto blob not null
) on commit preserve rows;
```

```
create global temporary table ts_laptop_f1(
  laptop_id number(10,0) constraint ts_laptop_f1_pk primary key,
  foto blob not null
) on commit preserve rows;
```

Prompt tablas temporales para la tabla servicio_laptop

```
create global temporary table ti_servicio_laptop_f1(
  num_servicio number(10, 0) not null,
  laptop_id number(10, 0) not null,
  importe number(8,2) not null,
  diagnostico varchar2(2000) not null,
```

```

factura blob,
sucursal_id number(10,0) not null,
constraint ti_servicio_laptop_f1_pk primary key (num_servicio, laptop_id)
);

--completar

```

- Observar que, para la tabla `laptop`, se crean 2 tablas temporales: La primera se crea para implementar transparencia con la instrucción `select` y la otra para implementar transparencia con la instrucción `insert`. Solo se crea para el fragmento `f1` ya que ahí se encuentra el dato BLOB.
- Notar que, para la tabla `servicio_laptop` se requieren 8 tablas temporales (4 para `insert`, 4 para `update`) debido al esquema de fragmentación horizontal derivado. En los 4 fragmentos se encuentra el dato BLOB.
- Observar que las tablas temporales empleadas para implementar transparencia para operaciones `select` solo requieren incluir la PK y la columna CLOB/BLOB mientras que las tablas temporales requeridas para implementar transparencia de operaciones `insert` incluyen todas las columnas.

1.1.2.4. Creación de funciones para acceso a datos CLOB/BLOB

Crear un script llamado `s-05-ilap-funciones-blob.sql`. El script contendrá la definición de las funciones requeridas para realizar acceso remoto a los datos CLOB/BLOB. Se requiere generar una función que extraiga el dato binario de cada fragmento.

Ejemplo

```

--@Autor:      Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción: Definición de funciones para acceso a BLOBs

Prompt creando función para extraer dato BLOB de foto_f1
create or replace function get_remote_foto_f1_by_id(p_laptop_id in number )
return blob is
pragma autonomous_transaction;
v_temp_pdf blob;
begin
--asegura que no haya registros
delete from ts_laptop_f1;
--Inserta los datos obtenidos del fragmento remoto a la tabla temporal.
insert into ts_laptop_f1
select laptop_id,foto
from laptop_f1

```



```

    where laptop_id = p_laptop_id;
--obtiene el registro de la tabla temporal y lo regresa como blob
select foto into v_temp_pdf
from ts_laptop_f1
where laptop_id = p_laptop_id;
--elimina los registros de la tabla temporal una vez que han sido obtenidos.
delete from ts_laptop_f1;
--termina txn autónoma.
commit;
return v_temp_pdf;
exception
when others then
    --si ocurre algún error, termina la txn autónoma.
    rollback;
    raise;
end;
/
show errors

```

Prompt creando función para extraer dato BLOB de servicio_laptop_f1

```

create or replace function get_remote_serv_lap_f1_by_id(
    p_num_servicio in number, p_laptop_id in number) return blob is
pragma autonomous_transaction;
v_temp_pdf blob;
begin
    --asegura que no haya registros
    delete from ts_servicio_laptop_f1;
    --Inserta los datos obtenidos del fragmento remoto a la tabla temporal.
    insert into ts_servicio_laptop_f1(num_servicio,laptop_id,factura)
        select num_servicio,laptop_id,factura
        from servicio_laptop_f1
        where num_servicio = p_num_servicio
        and laptop_id = p_laptop_id;
    --obtiene el registro de la tabla temporal y lo regresa como blob
    select factura into v_temp_pdf
    from ts_servicio_laptop_f1
    where num_servicio = p_num_servicio
    and laptop_id = p_laptop_id;
    --elimina los registros de la tabla temporal
    delete from ts_servicio_laptop_f1;
    --termina txn autónoma
exception
when others then
    --si ocurre algún error, termina la txn autónoma.

```

```

rollback;
raise;
end;
/
show errors
--completar

```

Observar que la función obtiene el dato binario del fragmento 1, y se emplea la tabla temporal `ts_laptop_f1`. Se deberán crear otras funciones similares para los nodos que requieran obtener el dato blob. Aplicar la misma técnica para las demás tablas globales que contengan datos binarios.

1.1.2.5. Creación de vistas con datos CLOB/BLOB

Crear un script por cada PDB llamado `s-05-ilap-<pdb>-vistas-blob.sql`. Cada script contendrá la definición de las vistas que contienen columnas con datos CLOB/BLOB. El script deberá hacer uso de las funciones creadas en la sección anterior únicamente cuando sea necesario hacer un acceso remoto para obtener un dato CLOB/BLOB.

Ejemplo

```

--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:    Definición de vistas con acceso a datos BLOB para el sitio
jrcbdd_s1
create or replace view laptop as
select laptop_id,num_serie,cantidad_ram,caracteristicas_extras,
       tipo_tarjeta_video_id,tipo_procesador_id,tipo_almacenamiento_id,
       tipo_monitor_id,laptop_reemplazo_id, get_remote_foto_f1_by_id(laptop_id) foto
from (
select laptop_id,num_serie,cantidad_ram,caracteristicas_extras,
       tipo_tarjeta_video_id,tipo_procesador_id,tipo_almacenamiento_id,
       tipo_monitor_id,laptop_reemplazo_id
from laptop_f2
union all
select laptop_id,num_serie,cantidad_ram,caracteristicas_extras,
       tipo_tarjeta_video_id,tipo_procesador_id,tipo_almacenamiento_id,
       tipo_monitor_id,laptop_reemplazo_id
from laptop_f3
union all
select laptop_id,num_serie,cantidad_ram,caracteristicas_extras,
       tipo_tarjeta_video_id,tipo_procesador_id,tipo_almacenamiento_id,
       tipo_monitor_id,laptop_reemplazo_id
from laptop_f4

```

```

union
select laptop_id,num_serie,cantidad_ram,caracteristicas_extras,
tipo_tarjeta_video_id,tipo_procesador_id,tipo_almacenamiento_id,
tipo_monitor_id,laptop_reemplazo_id
from laptop_f5
) q1;

create or replace view servicio_laptop as
select num_servicio,laptop_id,importe,diagnostico,factura,sucursal_id
from servicio_laptop_f1
union all
select num_servicio,laptop_id,importe,diagnostico,
get_remote_serv_lap_f2_by_id(num_servicio,laptop_id),
sucursal_id
from servicio_laptop_f2
union all
select num_servicio,laptop_id,importe,diagnostico,
get_remote_serv_lap_f3_by_id(num_servicio,laptop_id),
sucursal_id
from servicio_laptop_f3
union all
select num_servicio,laptop_id,importe,diagnostico,
get_remote_serv_lap_f4_by_id(num_servicio,laptop_id),
sucursal_id
from servicio_laptop_f4;
--Completar para Los demás sitios.

```

- En este ejemplo, la vista para la tabla `laptop` requiere obtener el dato BLOB del sitio remoto `arcbdd_s1`, por lo que requiere hacer uso de la función `get_remote_foto_f1_by_id`, sin embargo, la vista que se va a crear en el sitio `arcbdd_s1`, el dato BLOB (foto) es local, y por lo tanto no requiere hacer uso de la función.
- De forma similar, para la tabla `servicio_laptop`, el uso de la función dependerá de la ubicación del dato BLOB. En el ejemplo, el dato BLOB se encuentra en los 4 fragmentos, por lo tanto la vista hace uso de la función 3 veces (una vez para cada sitio remoto). El cuarto dato BLOB se obtiene de forma local es decir, el fragmento `servicio_laptop_f1`.

1.1.2.6. Ejecución de scripts de creación de vistas

Generar un archivo llamado `s-05-ilap-main-vistas.sql`. El script deberá conectarse a cada PDB y ejecutar los scripts anteriores.

Ejemplo

```
--@Autor:      Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción: Creación de vistas para todos los sitios

clear screen
whenever sqlerror exit rollback;

prompt =====
prompt Creando vistas para jrcbdd_s1
prompt =====

connect ilap_bdd/ilap_bdd@jrcbdd_s1
prompt creando vistas que no requieren manejo de BLOBs
@s-05-ilap-vistas.sql

prompt creando tablas temporales
@s-05-ilap-tablas-temporales.sql

prompt creando objetos para manejo de BLOBs
@s-05-ilap-funciones-blob.sql

prompt creando vistas con soporte para BLOBs
@s-05-ilap-jrc-s1-vistas-blob.sql

prompt =====
prompt Creando vistas para jrcbdd_s2
prompt =====

connect ilap_bdd/ilap_bdd@jrcbdd_s2
prompt creando vistas que no requieren manejo de BLOBs
@s-05-ilap-vistas.sql

prompt creando tablas temporales
@s-05-ilap-tablas-temporales.sql

prompt creando objetos para manejo de BLOBs
@s-05-ilap-funciones-blob.sql

prompt creando vistas con soporte para BLOBs
@s-05-ilap-jrc-s2-vistas-blob.sql
```

```
--completar
```

```
prompt Listo!
disconnect
```

Observar que se ejecutan los mismos scripts en todos los sitios excepto el último (marcado en negritas). Este último script contiene la definición de las vistas que contienen datos CLOB/BLOB y su definición es diferente para cada PDB.

1.2. Transparencia de distribución para instrucciones DML

El alcance del proyecto se limita a transparencia para instrucciones `insert` y `delete`

- Para implementar transparencia para instrucciones `insert`, se deberán crear `instead of triggers` para cada tabla fragmentada.
- Solo se deberá implementar los eventos de inserción y eliminación. El trigger deberá enviar un error cuando se intente realizar una operación `update`, indicando que estas aún no están implementadas.

Similar a la técnica vista en prácticas anteriores, se hará uso de triggers tipo `instead of` para implementar este nivel de transparencia.

1.2.1. Creación de triggers para tablas con esquema de fragmentación

En algunos casos, el código del trigger es distinto para cada sitio, por ejemplo, en fragmentaciones horizontales derivadas, o en tablas con datos CLOB/BLOB. En otros casos el código es el mismo. Con la finalidad de evitar duplicidad de código, realizar lo siguiente:

- Si el código es diferente para cada sitio, crear un archivo llamado `s-06-ilap-trigger-<pdb>-<tabla>.sql` para cada trigger
- Si el código es el mismo, crear un archivo llamado `s-06-ilap-trigger-<tabla>.sql`

Los triggers deberán lanzar una excepción cuando ocurra cada una de las siguientes situaciones:

Código de error	Descripción del error.
-20010	El registro que se intenta insertar o eliminar no cumple con el esquema de fragmentación horizontal primaria. Por ejemplo, suponer que los posibles valores para decidir el fragmento donde se aplicará la operación DML son A y B. Si el valor del campo es diferente a los 2 valores permitidos, el trigger deberá lanzar un error empleando este código.
-20020	El registro que se intenta insertar o eliminar no cumple con el esquema de fragmentación horizontal derivada. Este caso aplica cuando el trigger intenta ubicar al registro en el fragmento padre. Por ejemplo. Suponer que se desea insertar un registro de la tabla global <code>sucursal</code> que tiene una llave foránea <code>país_id =</code>

	1. El trigger intentará ubicar el fragmento donde se encuentra el registro padre cuya llave primaria tenga el valor <code>país_id = 1</code> . Si el registro no es localizado en algún nodo, el trigger deberá lanzar un error empleado este código.
-20030	Se intentó realizar una operación <code>update</code> en una tabla con esquema de fragmentación. Para propósitos del proyecto, esta operación no estará implementada. El trigger deberá lanzar un error indicando que operaciones <code>update</code> aún no se han implementado.

1.2.2. Creación de triggers para tablas replicadas

- Cada una de las vistas que representan a las tablas replicadas también deberán contar con su trigger de tipo `instead of`.
- A diferencia de las tablas con esquema de fragmentación, el trigger de cada tabla replicada si deberá implementar las 3 operaciones DML : `insert`, `update` y `delete`.
- En este caso el código del trigger no requiere lanzar ninguna de las 3 excepciones mencionadas anteriormente. La única excepción que puede generarse es la siguiente:

Código de error	Descripción del error.
-20040	Esta excepción se deberá lanzar cuando el código del trigger detecte que el registro no fue creado, actualizado o eliminado por alguna razón. Recordar que la BD puede regresar que se han afectado 0 registros, lo cual sería incorrecto ya que el trigger se dispara cuando se realiza una operación DML. Para los 3 tipos de operaciones, siempre se debe afectar un registro por réplica. En total 4 registros. La forma más simple para validar este requerimiento es empleando la instrucción <code>sql%rowcount</code> que indica el número de registros afectados de la última operación DML ejecutada. Ver el código de ejemplo más adelante.

1.2.2.1. Replicación síncrona de tablas

La técnica que se emplea para replicar las operaciones DML se le conoce como “replicación síncrona”. Esta estrategia tiene las siguientes ventajas y desventajas:

Ventajas

- Existe integridad y consistencia de datos en todo instante sin importar el número de réplicas.
- Al lanzar una operación DML sobre una tabla replicada, el trigger deberá ejecutar dicha instrucción en las 4 réplicas incluyendo la réplica local. Esto permite garantizar que las 4 réplicas conserven el mismo estado consistente en todo momento.

- En caso de ocurrir un error durante la aplicación de las operaciones DML entre las réplicas, se producirá una excepción la cual podrá ser manejada y en su caso generar un rollback. Esto garantizará regresar a un estado consistente de los datos.
- Fácil de implementar a través de un trigger.

Desventajas

- Cada operación DML requiere a su vez la aplicación de 4 operaciones. En el escenario donde existe una gran cantidad de operaciones DML sobre la tabla, el desempeño puede verse afectado.
- Si uno de los nodos presenta una falla o no está disponible, la operación DML no podrá ejecutarse, lo que afectaría directamente a la disponibilidad de la base de datos.

El esquema anterior es adecuado en situaciones donde existen pocas operaciones DML (baja frecuencia), y en situaciones donde se requiere integridad y consistencia en las N réplicas todo el tiempo. Estos requerimientos son los que se desean en este proyecto, por lo que se ha decidido emplearla. Existen otras técnicas, por ejemplo, sincronización asíncrona en la que es posible considerar exitosa una operación DML sin importar que en algunas de las réplicas no se haya realizado la operación. Las operaciones pendientes se programan para ser ejecutadas tiempo después. Oracle Golden Gate es un producto que permite realizar este tipo de esquemas de replicación.

De forma similar a las tablas con replicación, crear scripts `s-06-ilap-<tabla>-trigger.sql` para implementar los trigger `instead of`. Observar que para este tipo de tablas el código es exactamente el mismo en todos los nodos.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación:
--@Descripción:    trigger tipo_monitor

create or replace trigger t_dml_tipo_monitor
  instead of insert or update or delete on tipo_monitor
declare
  v_count number;
begin
  case
  when inserting then
    v_count := 0;
    --réplica local
    insert into tipo_monitor_r1(tipo_monitor_id,clave,descripcion)
    values (:new.tipo_monitor_id,:new.clave,:new.descripcion);
    v_count := v_count + sql%rowcount;
    --réplica 2
```

```
insert into tipo_monitor_r2(tipo_monitor_id,clave,descripcion)
values (:new.tipo_monitor_id,:new.clave,:new.descripcion);
v_count := v_count + sql%rowcount;
--réplica 3
insert into tipo_monitor_r3(tipo_monitor_id,clave,descripcion)
values (:new.tipo_monitor_id,:new.clave,:new.descripcion);
v_count := v_count + sql%rowcount;
--réplica 4
insert into tipo_monitor_r4(tipo_monitor_id,clave,descripcion)
values (:new.tipo_monitor_id,:new.clave,:new.descripcion);
v_count := v_count + sql%rowcount;

if v_count <> 4 then
    raise_application_error(-20040,
        'Número incorrecto de registros insertados en tabla replicada: '
        ||v_count);
end if;

when deleting then
    v_count := 0;
    --réplica local
    delete from tipo_monitor_r1 where tipo_monitor_id = :old.tipo_monitor_id;
    v_count := v_count + sql%rowcount;
    --réplica 2
    delete from tipo_monitor_r2 where tipo_monitor_id = :old.tipo_monitor_id;
    v_count := v_count + sql%rowcount;
    --réplica 3
    delete from tipo_monitor_r3 where tipo_monitor_id = :old.tipo_monitor_id;
    v_count := v_count + sql%rowcount;
    --réplica 4
    delete from tipo_monitor_r4 where tipo_monitor_id = :old.tipo_monitor_id;
    v_count := v_count + sql%rowcount;

    if v_count <> 4 then
        raise_application_error(-20040,
            'Número incorrecto de registros eliminados en tabla replicada: '
            ||v_count);
    end if;

when updating then
    --réplica local
```



```

v_count := 0;
update tipo_monitor_r1 set clave = :new.clave,descripcion =:new.descripcion
where tipo_monitor_id = :new.tipo_monitor_id;
v_count := v_count + sql%rowcount;
--réplica 2
update tipo_monitor_r2 set clave = :new.clave,descripcion =:new.descripcion
where tipo_monitor_id = :new.tipo_monitor_id;
v_count := v_count + sql%rowcount;
--réplica 3
update tipo_monitor_r3 set clave = :new.clave,descripcion =:new.descripcion
where tipo_monitor_id = :new.tipo_monitor_id;
v_count := v_count + sql%rowcount;
--réplica 4
update tipo_monitor_r4 set clave = :new.clave,descripcion =:new.descripcion
where tipo_monitor_id = :new.tipo_monitor_id;
v_count := v_count + sql%rowcount;
end case;

if v_count <> 4 then
  raise_application_error(-20040,
    'Número incorrecto de registros actualizados en tabla replicada: '
    ||v_count);
end if;

end;
/
show errors

```

Notar la lógica para validar la excepción con código -20040.

Al terminar de crear los scripts anteriores, invocarlos en uno nuevo llamado [s-06-ilap-main-triggers.sql](#). Tener cuidado con invocar cada script con base al sitio donde deben ejecutarse.

Ejemplo

```

--@Autor:      Jorge A. Rodríguez C
--@Fecha creación: 13/04/2017
--@Descripción:  Creación de trigger para implementar transparencia de
inserción
clear screen
whenever sqlerror exit rollback

```

```
Prompt =====
Prompt Creando triggers en jrcbdd_s1
Prompt =====
connect ilap_bdd/ilap_bdd@jrcbdd_s1
@s-06-ilap-sucursal-trigger.sql
@s-06-ilap-jrc-s1-sucursal-taller-trigger.sql
@s-06-ilap-jrc-s1-sucursal-venta-trigger.sql
@s-06-ilap-laptop-trigger.sql
@s-06-ilap-laptop-inventario-trigger.sql
@s-06-ilap-historico-status-laptop-trigger.sql
@s-06-ilap-jrc-s1-servicio-laptop-trigger.sql
@s-06-ilap-tipo-procesador-trigger.sql
@s-06-ilap-tipo-almacenamiento-trigger.sql
@s-06-ilap-tipo-monitor-trigger.sql
@s-06-ilap-tipo-tarjeta-video-trigger.sql

Prompt =====
Prompt Creando triggers en jrcbdd_s2
Prompt =====
connect ilap_bdd/ilap_bdd@jrcbdd_s2
@s-06-ilap-sucursal-trigger.sql
@s-06-ilap-jrc-s2-sucursal-taller-trigger.sql
@s-06-ilap-jrc-s2-sucursal-venta-trigger.sql
@s-06-ilap-laptop-trigger.sql
@s-06-ilap-laptop-inventario-trigger.sql
@s-06-ilap-laptop-inventario-trigger.sql
@s-06-ilap-historico-status-laptop-trigger.sql
@s-06-ilap-jrc-s2-servicio-laptop-trigger.sql
@s-06-ilap-tipo-procesador-trigger.sql
@s-06-ilap-tipo-almacenamiento-trigger.sql
@s-06-ilap-tipo-monitor-trigger.sql
@s-06-ilap-tipo-tarjeta-video-trigger.sql

Prompt =====
Prompt Creando triggers en arcbdd_s1
Prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s1
@s-06-ilap-sucursal-trigger.sql
@s-06-ilap-arc-s1-sucursal-taller-trigger.sql
@s-06-ilap-arc-s1-sucursal-venta-trigger.sql
@s-06-ilap-laptop-trigger.sql
@s-06-ilap-laptop-inventario-trigger.sql
@s-06-ilap-historico-status-laptop-trigger.sql
@s-06-ilap-arc-s1-servicio-laptop-trigger.sql
```

```
@s-06-ilap-tipo-procesador-trigger.sql
@s-06-ilap-tipo-almacenamiento-trigger.sql
@s-06-ilap-tipo-monitor-trigger.sql
@s-06-ilap-tipo-tarjeta-video-trigger.sql

Prompt =====
Prompt Creando triggers en arcbdd_s2
Prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s2
@s-06-ilap-sucursal-trigger.sql
@s-06-ilap-arc-s2-sucursal-taller-trigger.sql
@s-06-ilap-arc-s2-sucursal-venta-trigger.sql
@s-06-ilap-arc-s2-laptop-trigger.sql
@s-06-ilap-laptop-inventario-trigger.sql
@s-06-ilap-historico-status-laptop-trigger.sql
@s-06-ilap-arc-s2-servicio-laptop-trigger.sql
@s-06-ilap-tipo-procesador-trigger.sql
@s-06-ilap-tipo-almacenamiento-trigger.sql
@s-06-ilap-tipo-monitor-trigger.sql
@s-06-ilap-tipo-tarjeta-video-trigger.sql

Prompt Listo!
```

- Observar que solo los triggers correspondientes a `sucursal`, `laptop_inventario` e `histórico_status_laptop` se pueden reutilizar en todos los nodos.
- El trigger de `laptop` se reutiliza en 3 nodos y en el nodo donde se encuentra el dato BLOB se realiza un acceso local.

Continuar con la siguiente parte del proyecto.